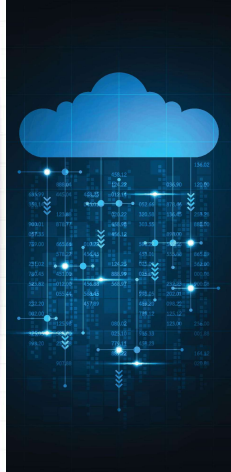




Wrocław University of Science and Technology



Cloud programming

Rafał Palak

Wrocław University of Science and Technology

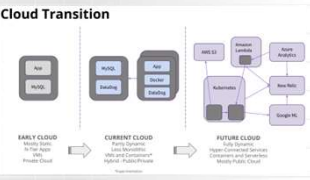
1



Wrocław University of Science and Technology

Infrastructure as Code (IaC)[1]

- **Management and sharing** of infrastructure through **code**, not manual processes
- **Configuration files** are created containing **infrastructure specifications**, which facilitates editing and distribution of configurations
- **The same environment is created each time**
- By **codifying and documenting configuration specifications**, IaC supports **configuration management** and helps **avoid undocumented, ad hoc configuration changes**



Cloud Transition

EARLY CLOUD: Manual provisioning, on-premise hardware, manual configuration, manual updates

CURRENT CLOUD: Fully managed services, on-premise hardware, manual configuration, manual updates

FUTURE CLOUD: Fully managed services, on-premise hardware, manual configuration, manual updates

2



Wrocław University of Science and Technology

Infrastructure as Code (IaC)[2]

- **Version control** is an important part of IaC,
- **Configuration files** should be treated **just like any other software source code file**.
- **Implementing infrastructure as code** also means you can **break your infrastructure into modular components**, which can then be combined in various ways through automation.
- Without IaC, it is becoming **increasingly difficult to manage the scale of today's infrastructure**.
- It **improves consistency** and **reduces errors and manual configuration**.



I deployed to production from my local machine

Use git!

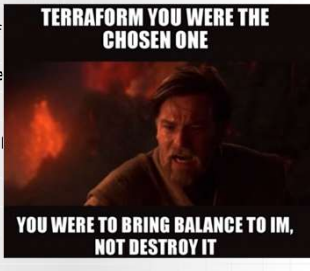
3



Wrocław University of Science and Technology

Terraform

- One of the **more popular tools** for managing infrastructure as code (IaC)
- Allows to **describe the structure** of your infrastructure using the **HashiCorp Configuration Language (HCL)** or **JSON**
- An **open-source project** with **incredible flexibility**, supporting all the most popular cloud platforms



TERRAFORM YOU WERE THE CHOSEN ONE

YOU WERE TO BRING BALANCE TO IM, NOT DESTROY IT

4



Wrocław University of Science and Technology

CloudFormation

- Works **only for AWS**
- **Fully integrated with AWS**, allowing for greater control
- Allows you to describe the structure of your infrastructure using **YAML and JSON**
- **Offers rollback** enabling the application to be restored to its previous state
- **The close relationship of this tool with AWS** enables the deployment of infrastructure across multiple regions and accounts using the same CloudFormation template.
- Competitors' equivalents: **Google Cloud Deployment Manager** and **Azure Resource Manager**



1. Create a template file (YAML or JSON) describing the infrastructure you want to deploy.

2. Use the CloudFormation CLI or console to create a new stack or update an existing one.

3. CloudFormation creates the infrastructure defined in your template.

4. CloudFormation monitors the progress of the stack creation and reports the status.

5. CloudFormation provides a web console to view the status of your stack and its resources.

5



Wrocław University of Science and Technology

Terraform - basics

6



Teraform initialization - terraform init [1]

- Downloading Providers
 - Terraform uses various providers to interact with different cloud services or technologies.
 - Terraform automatically downloads the necessary provider plugins that are listed in the configuration.
- Creating the .terraform Directory
 - Terraform creates a .terraform directory where it stores downloaded provider plugins and other temporary files that are used to manage the state of your infrastructure.



Terraform WORKFLOW

Init → Plan → Apply → Destroy

7

7



Teraform initialization - terraform init [2]

- Setting Up the Backend
 - If a backend (used for storing the state of the infrastructure) is defined in your configuration, 'terraform init' will configure it accordingly.
 - Backends can be used to store information about the infrastructure's state remotely, facilitating collaboration and management of infrastructure in teams.
- Initializing State
 - Terraform initializes the infrastructure's state by creating a new state file or configuring access to an existing state depending on the configuration.



Terraform WORKFLOW

Init → Plan → Apply → Destroy

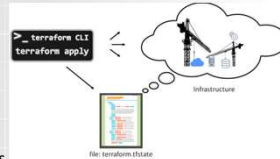
8

8



State [1]

- Refers to the current configuration and properties of the infrastructure resources that are managed by Terraform.
- The state file is used to map resources in the Terraform configuration to actual resources in the cloud service.
- Allows Terraform to track information about the managed resources, which is crucial for proper planning and making changes to the infrastructure.
- Enables Terraform to know which resources exist and what their current properties are.



File: terraform.tfstate

9

9



State [2]

- Crucial for avoiding conflicts and inconsistencies when different people or systems make changes to the infrastructure.
- Allows Terraform to optimize operations, only making changes where they are truly needed, instead of recreating everything from scratch each time.
- Used to display output values after applying changes, which facilitates easy sharing of information between different Terraform modules and configurations.



Accidentally deletes Terraform State File Without any backups

INFRASTRUCTURAL DAMAGE

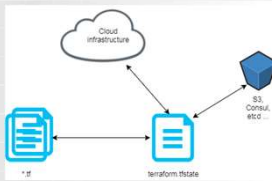
10

10



State [3]

- Terraform allows for the management of state both locally and remotely.
- Storing the state remotely is recommended for larger teams and infrastructures, as it provides better control and security.
- The state file is typically a JSON file that can be viewed and managed using various Terraform commands, such as 'terraform state list' or 'terraform state show'.



Cloud Infrastructure

terraform.tfstate

S3, Consul, etc.

11

11



Planning Changes – terraform plan

- One of the most important tools in the Terraform ecosystem.
- It allows for an analysis of the differences between the current state of the infrastructure (stored in the state file) and the state described in the configuration files.
- Based on this analysis, Terraform generates an action plan that shows which resources will be created, modified, or deleted.
- The plan is displayed in the console, giving the user a full overview of the proposed changes before they are actually applied.



Terraform WORKFLOW

Init → Plan → Apply → Destroy

12

12

Planning Changes – why?

- Security - Enables a secure review of what changes will be made before they are actually applied. This helps avoid unwanted side effects.
- Accuracy - Helps in identifying errors in the configuration or discrepancies with the actual infrastructure before attempting to make changes.
- Control - Provides the ability to review and approve changes by other team members or through automated deployment processes (CI/CD).
- Documentation - The generated plan can serve as documentation of the changes, which can be archived for audit purposes or analysis.



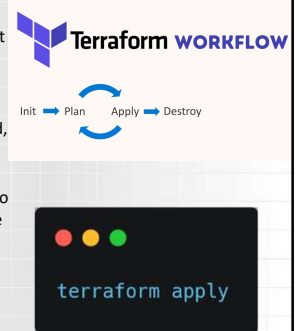
```
terraform plan
```

13

13

Applying Changes – `terraform apply` [1]

- Used to make changes to the infrastructure according to the current configuration described in the Terraform files.
- First, it generates a change plan that shows which resources will be created, modified, or deleted.
- After generating the plan, the command asks the user for approval to apply the changes. You can review the plan and decide whether to apply it.



14

14

Applying Changes – `terraform apply` [2]

- After approval, `terraform apply` actually executes the changes in the infrastructure: it creates, modifies, or deletes resources according to the plan.
- Upon completion of the operation, the state of the infrastructure is updated to reflect the changes made.



15

15

Destroying Resources – `terraform destroy`

- Removes all resources described in the Terraform configuration files that were previously created by `terraform apply`.
- Similar to `terraform apply`, the `destroy` command also asks for confirmation before executing.
- Allows you to see which resources will be removed and decide whether to proceed.
- State Update After the operation is completed, the state file is updated to reflect the absence of resources.



16

16

Destroying Resources – why?

- Releasing Resources - If certain infrastructure is no longer needed, it can be easily removed, which can be particularly important in the context of cost management.
- Cleaning Up Environments - This is useful in testing and development environments, where infrastructure is often created and destroyed.
- Security - The confirmation before deletion provides additional control over what will be destroyed.
- Automation - The resource destruction process can be automated as part of a CI/CD workflow, though it requires caution to avoid deleting important resources.



```
terraform destroy
```

17

17

Destroying Resources – Caution!

- In production environments - Always make sure you know which resources will be removed and consider the consequences of their deletion.
- Use the `-target` flag to specify particular resources you want to destroy, instead of destroying everything. For example: `terraform destroy -target=aws\_instance.my\_instance`.



18

18



State and How It Is Managed

- State management is one of the key aspects of working with Terraform.
- The state in Terraform is a record that stores information about the resources managed by Terraform in a given project.
- The state file is used by Terraform to map resources in the configuration to actual resources in the target environment.



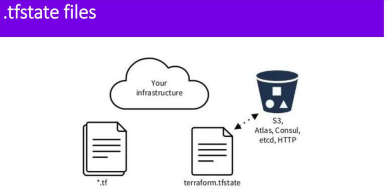
19

19



State - Storage

- Locally - By default, Terraform stores the state in a file named 'terraform.tfstate' in the working directory. This is suitable for simple, individual projects.
- Remotely - For teams and more complex projects, storing the state remotely is recommended. This allows for better access management, security, and change history.
- Terraform supports multiple backend options for storing the state, including services such as AWS S3, Google Cloud Storage, Azure Blob Storage, etc.



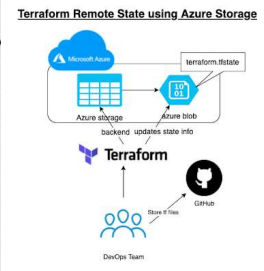
20

20



State - the structure of the state file

- The state file in JSON format stores a wide range of information about the managed resources.
 - Resource Identifiers - Unique identifiers of resources in the service provider.
 - Resource Attributes - The current attributes of each resource, e.g., the IP address of a VM instance.
 - Dependencies between Resources - Information on how resources are dependent on each other.



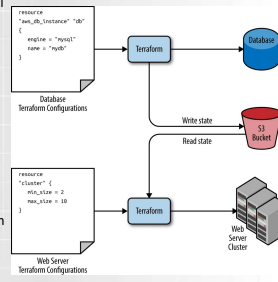
21

21



State and How It Is Managed

- Security: The state contains sensitive data, such as passwords or API keys in an encrypted form. Always secure access to state files and use remote backends with appropriate access control and encryption measures.
- State Locking - To prevent conflicts, Terraform allows for "locking" the state during the application of changes, preventing simultaneous modifications by different processes.
- Version Management - Some backends (e.g., S3 with versioning enabled) allow for storing multiple versions of the state file, which facilitates easy restoration to a previous state.
- State as Environment - The ability to manage multiple environments (e.g., production, staging) by creating separate state configurations for each of them.



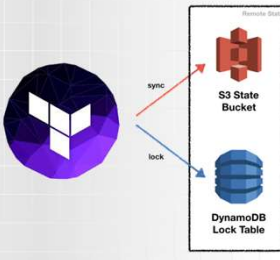
22

22



State - Best Practices

- Avoid Manual Editing - Direct editing of state files is risky and can lead to inconsistencies. It is recommended to use Terraform commands to manage the state.
- Regular Reviews and Backups - Regularly check and back up state files to avoid data loss.
- Automation - Automate state-related processes such as backups or migrations between environments as much as possible.
- Proper state management is key to using Terraform effectively and securely in infrastructure management.



23

23



Thank you for your attention

24

24